

MetraVerify: Full Project Report

1. Executive Summary

MetraVerify is a comprehensive, full-stack digital trust platform built for the lifecycle management of weighing and measuring instruments under Legal Metrology standards. The platform acts as a secure, role-driven, multi-tier web application, seamlessly orchestrating interactions between Business Users, Legal Metrology Officers (LMO), Government Approved Test Centres (GATC), and Administrative staff. By digitizing workflows ranging from instrument registration to certification and expiry management, MetraVerify enhances transparency, security, and operational efficiency in statutory processes.

2. Project Architecture and Technology Stack

The project adheres to a modern decoupled architecture, split between a robust Node.js/Express REST API backend and a responsive React Single Page Application (SPA) frontend.

2.1 Backend (Server-Side)

- **Runtime Environment:** Node.js (ESM Modules)
- **Framework:** Express.js v4
- **Database:** MongoDB (NoSQL) with Mongoose ORM for schema definition and validation.
- **Authentication:** Stateless JSON Web Tokens (JWT) signed with HMAC-SHA256, supported by bcryptjs for password hashing (Salt work factor of 10).
- **File Handling & Processing:** Multer for file upload filtering and sanitization.
- **Certificate Generation:** PDFKit for in-flight, dynamic rendering of high-fidelity PDF certificates (streamed directly without disk storage).
- **QR Code Engine:** qrcode library to generate tamper-evident cryptographic QR codes linking to public verification portals.
- **Security:** CORS, Morgan HTTP Logger, Centralized Error Handling, and RBAC (Role-Based Access Control) middlewares. All secrets are externalized via .env .

2.2 Frontend (Client-Side)

- **Framework:** React 18 (Vite Bundler for fast builds and hot module replacement)
- **Routing:** React Router v6, implementing protected, role-guarded routes.
- **Styling:** Tailwind CSS for a scalable, government-grade utility-first design system.
- **Icons & Visualization:** Lucide React for modern iconography, and Recharts for interactive analytics and dashboards.
- **Networking:** Axios with request/response interceptors to handle JWT Bearer injection and automatic 401 Unauthorized handling.

3. Core Subsystems and Workflows

3.1 Role-Based Access Control (RBAC)

MetraVerify supports a rigid access hierarchy:

- **Public:** Can access the landing page, register/login, and utilize the unauthenticated QR code scanning portal for verifying certificates.
- **Business Users:** Can register instruments, submit verification applications, and view their certificates/dashboards.

- **LMO (Legal Metrology Officers):** Can access assigned applications, conduct 7-point inspections, issue/revoke certificates.
- **GATC (Government Approved Test Centres):** Handle lab test queues and specialized calibration metrics.
- **Admin:** Oversee the entire system, manage users, monitor analytics dashboards, and assign applications.

3.2 7-Point Statutory Inspection Engine

The system enforces a strict 7-point check for instrument verification:

1. **Instrument Condition:** Visual and mechanical stability.
2. **Display:** Readability of the digital/analog output.
3. **Zero Error:** Return-to-zero capabilities.
4. **Accuracy:** Tested against certified standard weights.
5. **Calibration:** Repeatability and hysteresis checks.
6. **Sealing/Stamping:** Application of physical tamper-evident seals.
7. **Physical Condition:** Operating environment controls.

Integrity Rule: A certificate can only be issued if all 7 parameters pass. Any failure automatically rejects the case with statutory remarks.

3.3 Digital Certificate and QR Code Service

- Passing an inspection triggers the automatic generation of a `Certificate` document featuring a unique sequence identifier (e.g., `CERT-YYYY-XXXXXX`).
- A clean-URL QR code is generated (e.g., `http://<domain>/verify/<cert_id>`), ensuring no private business data or Personally Identifiable Information (PII) is embedded directly into the payload.
- Certificates are rendered instantly as PDFs upon request.

3.4 Real-Time Expiry Management

Certificates possess built-in lifecycles monitored dynamically:

- **VALID (Green):** > 30 days remaining.
- **EXPIRING_SOON (Amber):** ≤ 30 days remaining.
- **EXPIRED (Red):** Validity date passed.
- **REVOKED (Black/Struck):** Manually revoked due to fraud or tampering.

4. Security, Audit, and Data Models

4.1 Audit Trailing

Every sensitive state mutation within the application (e.g., user registrations, application submissions, inspections, certificate issuance/revocations) triggers an immutable record in the `AuditLog` collection. These logs capture actor metadata, entity references, IP addresses, and exact timestamps, ensuring total compliance and accountability.

4.2 Data Relationships

The MongoDB schemas are heavily normalized via Mongoose references:

- **Users** own **Instruments**.
- **Users** submit **Verification Applications** (referencing specific Instruments).

- **Applications** trigger **Inspections**.
- Successful **Inspections** yield **Certificates**.
- **Audit Logs** and **Notifications** are generated organically across these entity interactions.

5. System Dashboards

The application provides customized analytics dashboards powered by **Recharts** :

- **Business Dashboard:** Views of expiring-soon alerts, monthly compliance trends, and active instruments.
- **LMO Dashboard:** Daily queues, pending inspections, and completion rates.
- **Admin Dashboard:** Holistic system view encompassing caseload distribution and global user statistics.

6. Conclusion

The MetraVerify system is a prime example of standardizing and securing government or industrial compliance workflows. Through modern architecture (React + Node + Mongo), stringent Role-Based Access Control, live PDF rendering, and comprehensive audit logs, the platform drastically reduces paperwork, mitigates fraud, and brings Legal Metrology into the digital age.